

Reinforcement Learning

Solving Blackjack and Lunar Lander (Continuous) Environments

Project done by:

Elton Silva, nº 20231976

1. Introduction

This project explores two different Reinforcement Learning (RL) environments using the Gymnasium library, with the goal of developing agents that can learn optimal policies through interaction. This project will focus on two environments, one discrete (Blackjack) and another with continuous observation/action spaces (Lunar Lander). The project implements and compares the performance of multiple RL algorithms, providing insights into their convergence behavior, policy effectiveness, and reward optimization.

The following notebooks were used:

- Notebook 1: Blackjack.ipynb focuses on the Blackjack-v1 environment, which has a discrete state-action space. The algorithms used were Double Q-Learning and Expected SARSA.
- Notebook 2: Lunar_Lander.ipynb targets a more complex, continuous environment, requiring advanced algorithms Deep RL solutions suitable for continuous control tasks. The algorithms used were Proximal Policy Optimisation (PPO) and Soft-Actor Critic (SAC).

This report shall start by explaining the environments picked. The following section will be dedicated to explaining the workings of each algorithm. Then the approach used for the problems will be detailed. Finally, the results achieved will be discussed and the conclusions derived from this project will be presented.

2. Environments Description

2.1. Blackjack

The Blackjack environment is a simple, discrete, turn-based environment that simulates the game of 21 (Blackjack), a well-known card game played between a player and a dealer. It is often used as an introductory benchmark for RL algorithms because of its:

- Finite state and action space
- Built-in randomness
- Well-understood optimal policy

It provides an ideal setting to study value-based methods like Monte Carlo control, SARSA, and Q-learning.

2.1.1. Action Space

The action shape is (1,) in the range {0, 1} indicating whether to stick or hit.

- 0: Stick (End the player's turn.)
- 1: Hit (Take another card from the deck)

2.1.2. State Space

The state space is relatively small and enumerable, which makes it easy to apply tabular methods:

- Player sum: 12–21 (below 12 the player will always hit automatically)
- Dealer showing: 1–10
- Usable ace: True / False

Usable Ace explained:

A "usable ace" refers to the fact that the ace can count as 11 without the player busting. If the player's total exceeds 21 and a usable ace is present, it is automatically converted from 11 to 1. This mechanic is what makes the environment non-trivial, because it changes the value function nonlinearly depending on the presence of usable aces.

This gives a total of:

$$(10 \text{ player sums}) \times (10 \text{ dealer cards}) \times (2 \text{ ace flags}) = 200 \text{ states}$$

2.1.3. Rewards

The reward system is the following:

- +1 if the player wins (dealer busts or player has higher score)
- 0 if it's a draw (equal scores)
- -1 if the player loses (player busts or dealer has higher score)

2.1.4. Termination Condition

The episode ends if the player hits and the sum of hand exceeds 21 or the player sticks.

2.2. Lunar Lander

The LunarLander-v3 (continuous) environment is a simulated control task from the Box2D physics engine, available in the gymnasium library. It represents a simplified 2D version of the classic spacecraft landing task with the goal to land a rocket module safely on the surface, subject to realistic physics.

It is a continuous control problem, the action space is a continuous vector, the dynamics are non-linear, time-sensitive and physics based. The agent outputs real-valued forces that are mapped to physical actuators (like thrust and side engines). It is systematically episodic and fully observable (the agent receives the complete state of the lander at each timestep).

Configuration:

- **continuous=True**: Uses a **continuous action space** — the agent outputs real-valued thrust commands.
- **gravity=-10.0**: Modifies gravitational acceleration (normally -10.0 m/s²).
- **enable_wind=True**: Enables environmental stochasticity via wind forces.
- **wind_power=15.0**: Sets the strength of wind effects.
- **turbulence_power=1.5**: Adds small random force fluctuations.

2.2.1. Action Space

In the discrete version, there are four discrete actions available [5]:

- 0: do nothing
- 1: fire left orientation engine
- 2: fire main engine
- 3: fire right orientation engine

In the continuous version, these actions are not discrete choices like "fire left engine" or "do nothing", but real-valued inputs that simulate analog engine control — requiring more nuanced policies and benefiting from methods like **PPO** or **SAC**. In continuous mode, the action vector is two-dimensional.

For Value 1 [4]:

- [-1.0, +1.0] for main engine where [-1.0, 0.0] = Off and [0.0, +1.0] = On

For Value 2 [5]:

- [-1.0, -0.5]: Left Engine
- [-0.5, 0.5]: Off

- [0.5, 1.0]: Right Engine

2.2.2. State Space

Returning to the state space, a vector of 8 continuous values is returned to the agent at each timestep. This vector represents the state of the lander and its interaction with the environment. This vector fully specifies the system's configuration and is necessary to make informed decisions.

1. x-position
2. y-position
3. x-velocity
4. y-velocity
5. lander angle
6. angular velocity
7. left leg contact with ground (boolean)
8. right leg contact with ground (boolean)

2.2.3. Rewards

After every step a reward is granted. The total reward of an episode is the sum of the rewards for all the steps within that episode.

For each step, the reward:

- is increased/decreased the closer/further the lander is to the landing pad.
 - is increased/decreased the slower/faster the lander is moving.
 - is decreased the more the lander is tilted (angle not horizontal).
 - is increased by 10 points for each leg that is in contact with the ground.
 - is decreased by 0.03 points for each frame a side engine is firing.
 - is decreased by 0.3 points for each frame the main engine is firing.
- The episode receives an additional reward of -100 or +100 points for crashing or landing safely respectively.

An episode is considered a solution if it scores at least 200 points. [4]

2.2.4. Termination Conditions

An episode ends under the following conditions:

- The lander touches the ground (either it crashes or achieves a soft landing).
- The lander goes out of bounds (too far left, right, or above).
- The episode exceeds the maximum number of steps (around 1000 by default).

Termination is deterministic and clearly signals success or failure, which simplifies the credit assignment problem for reinforcement learning algorithms.

3. Algorithms Description

3.1. Double Q-Learning

Double Q-Learning is an Off-policy, value-based method with a focus on reducing overestimation bias inherent in standard Q-learning.

This variation of Q-Learning reduces overestimation bias by maintaining two separate Q-tables (Q1 and Q2) where updates alternate randomly between the two tables, using one to select actions and the other to

evaluate them. This decouples action selection and evaluation, significantly reducing overestimation. The networks are often alternated in their roles, or both are trained symmetrically.

$$\begin{aligned}
 & \text{if } \text{UPDATE}(A) \text{ then} \\
 & \quad \text{Define } a^* = \operatorname{argmax}_a Q^A(s', a) \\
 & \quad Q^A(s, a) \leftarrow Q^A(s, a) + \alpha(s, a) (r + \gamma Q^B(s', a^*) - Q^A(s, a)) \\
 & \text{else if } \text{UPDATE}(B) \text{ then} \\
 & \quad \text{Define } b^* = \operatorname{argmax}_a Q^B(s', a) \\
 & \quad Q^B(s, a) \leftarrow Q^B(s, a) + \alpha(s, a) (r + \gamma Q^A(s', b^*) - Q^B(s, a))
 \end{aligned}$$

Figure 1 - Double Q-Learning's Update Rule [1]

- Exploration strategy: Epsilon-greedy with exponential decay.
- Update rule: Temporal Difference (TD) learning using best-next action from the alternate Q-table.
- Performance analysis: Returns per episode and convergence metrics plotted with moving averages and histograms.

Standard Q-learning updates are known to overestimate Q-values due to the use of the max operator:

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t(s_t, a_t) (r_t + \gamma \max_a Q_t(s_{t+1}, a) - Q_t(s_t, a_t))$$

Figure 2 - Q-Learning's Update Rule [1]

This overestimation happens because the same Q-values are used both to select and evaluate the best action, which amplifies any positive noise in the estimates.

Double Q-learning is foundational to modern deep RL algorithms like Double DQN and TD3, where it plays a critical role in maintaining value estimation accuracy.

3.2. Expected SARSA

Expected SARSA is an On-policy, value-based method with the main goal of providing a smoother and less biased Q-value update than traditional SARSA.

SARSA (State-Action-Reward-State-Action) updates its Q-values using the action taken in the next state, making it sensitive to the agent's behavior policy. Expected SARSA improves upon this by updating the Q-values using the expected value over all possible next actions under the current policy.

The update rule is:

$$\begin{aligned}
 Q(s_t, A_t) & \leftarrow Q(s_t, A_t) + \alpha [R_{t+1} + \gamma E_{\pi}[Q(s_{t+1}, A_{t+1}) | s_{t+1}] - Q(s_t, A_t)] \\
 & \leftarrow Q(s_t, A_t) + \alpha [R_{t+1} + \gamma \sum_a \pi(a | s_{t+1}) Q(s_{t+1}, a) - Q(s_t, A_t)]
 \end{aligned}$$

Figure 3 - Expect SARSA's Update Rule [2]

This expectation makes Expected SARSA:

- Less stochastic than standard SARSA (since it averages over actions).
- More stable, especially when the policy is stochastic.
- Still on-policy, because it uses the current behavior policy π in the update.

- Expected SARSA is a midway point between SARSA (which is biased but low-variance) and Q-learning (which is off-policy and can have higher variance but less biased).

3.3. Proximal Policy Optimisation (PPO)

PPO is an On-policy, policy gradient method with the objective of improving training stability of policy gradient methods while keeping high performance.

PPO belongs to the family of policy optimization algorithms, where the goal is to directly learn the policy $\pi(a_t | s_t; \theta)$ that maps states to action probabilities. It improves upon earlier methods like Trust Region Policy Optimization (TRPO) by simplifying the implementation and reducing computational overhead.

Policy gradient methods can be unstable because large updates to the policy can drastically reduce performance. PPO addresses this using a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min (r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)]$$

Figure 4 – PPO's clipped surrogate objective [6]

Where:

- $r_t(\theta) = \pi_{\theta}(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ is the ratio of the new to old policy probabilities.
- $\hat{A}_t$ is the estimated advantage function.
- The clipping operation prevents r_t from moving too far from 1 (i.e., discourages large changes to the policy).

This constrains policy updates, preventing drastic shifts that could degrade performance, but still allows for efficient learning. PPO strikes a balance between performance and stability and has become a default choice for many continuous control problems.

3.4. Soft-Actor Critic (SAC)

SAC is an Off-policy, actor-critic method that encourages exploration and improve sample efficiency using entropy maximization.

SAC is based on the maximum entropy reinforcement learning framework, which augments the standard objective by adding an entropy bonus. Instead of maximizing expected returns alone, it maximizes:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \pi} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))]$$

Figure 5 - SAC's Maximum Entropy Objective Function [7]

Where:

- $\mathcal{H}(\pi(\cdot | s_t)) = -\mathbb{E}_{a_t \sim \pi} [\log \pi(a_t | s_t)]$ represents the entropy of the policy.
- α is a temperature parameter that controls the trade-off between exploration (entropy) and exploitation (reward).

4. Methodology

4.1. Blackjack

The Blackjack environment presents a stochastic, low-dimensional control problem with a clear goal: maximizing expected return through optimal policy learning under uncertainty (e.g., hidden dealer card, chance of busting). Double Q-Learning was chosen to address the environment's tendency to overvalue high-variance actions, using two value tables to reduce overestimation. Expected SARSA was selected to smooth learning by incorporating the full action distribution under the current policy, reducing volatility from abrupt

max-based updates. Deep RL methods were unnecessary due to the small state space, and simpler methods like basic Q-learning were avoided for their instability in noisy reward settings.

4.2. Lunar Lander

Algorithm Selection and Justification

As mentioned above, to solve this environment, two reinforcement learning algorithms were implemented, Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC)

Each algorithm was chosen based on the environment's requirements and the underlying theoretical strengths of the methods.

4.2.1. Proximal Policy Optimization (PPO)

PPO is an on-policy policy gradient algorithm that improves training stability over traditional policy gradient methods by introducing a clipped surrogate objective function. This constraint prevents policy updates from deviating too far from the previous policy, which is critical in high-variance settings like continuous control.

Why PPO was used:

- It handles continuous action spaces natively.
- It has proven performance in simulated physics environments (e.g., MuJoCo, Box2D).
- Its training stability is valuable in complex environments where unbounded updates could derail convergence.
- Entropy regularization embedded in PPO encourages adequate exploration without overwhelming exploitation.

Theoretically, this algorithm provides a well-balanced approach between learning speed and robustness, especially important under environmental noise (like wind and turbulence in this environment).

4.2.2. Soft Actor-Critic (SAC)

SAC is an off-policy actor-critic algorithm grounded in maximum entropy reinforcement learning. It explicitly adds an entropy term to the reward function, encouraging exploration by favoring stochastic policies. Unlike PPO, which is on-policy, SAC can reuse past experiences, making it highly sample efficient.

Why SAC was used:

- Its entropy-based objective makes it well-suited to environments with stochastic dynamics and continuous control, like LunarLander-v3.
- It maintains a stochastic policy throughout training, which is useful in environments where deterministic actions could fail under minor perturbations (e.g., wind).
- Its off-policy nature allows more efficient learning from past experiences, making it more data-efficient compared to PPO.

SAC is particularly tailored to projects like this due to the continuous action space, the need for robust policies, and the challenge of generalizing across different episodes with environmental randomness.

4.2.3. PPO Agent Design

The PPO agent was designed using an actor-critic architecture. A shared neural network was used to extract state features, which then branched into two heads: one for predicting the action distribution (actor) and another for estimating the expected return from a state (critic).

Actions were sampled from a Gaussian distribution parameterized by the actor network and then scaled to match the environment's required action range. A key component of PPO is its ability to maintain training stability by limiting the size of policy updates. This was achieved by comparing the probability of taking an action under the current policy versus the old policy and clipping the update when this change became too large.

During training, the agent interacted with the environment to collect trajectories consisting of states, actions, rewards, and value predictions. These were then used to compute return estimates and train the policy. The PPO implementation used batch updates and multiple epochs per training cycle, which helped ensure thorough learning from each batch of collected experience. Additionally, an entropy bonus was applied to encourage exploratory behavior during early training.

Logging mechanisms tracked total reward per episode, and evaluation runs were periodically conducted to verify that the agent was learning a generalizable policy. The performance was assessed based on how smoothly and reliably the lander could descend and touch down within the landing zone.

4.2.4. SAC Agent Design

The SAC agent followed a more modular design, using distinct neural networks for each component: two critics to estimate action-values, a separate actor to produce a stochastic policy, and a target network to stabilize training. SAC was chosen for its ability to handle high-variance environments efficiently through entropy maximization and off-policy learning.

Unlike PPO, SAC does not rely on freshly collected trajectories for each update. Instead, it maintains a large replay buffer that stores experiences from past interactions. This allows it to sample batches randomly and reuse data more effectively, significantly improving sample efficiency.

The SAC policy was also stochastic, producing both a mean and a standard deviation for a Gaussian action distribution. However, it introduced additional control by automatically adjusting the policy's entropy over time. This meant the agent could explore more broadly during early training and gradually become more deterministic as it improved.

Each component of the SAC agent was trained separately. The critic networks were updated by minimizing the difference between predicted and target Q-values, while the actor was trained to maximize expected return while preserving high entropy. The use of two Q-networks helped reduce the overestimation bias common in value-based methods.

Training involved continual interaction with the environment, storing transitions in the replay buffer, and sampling mini-batches for updates. Regular soft updates of the target networks ensured training remained stable despite the high capacity and complexity of the models.

5. Evaluation

5.1. Blackjack

Contrary to ideal expectations, neither algorithm converges to a stable policy. Learning curves are unstable and oscillate even with smoothing, indicating inconsistent performance. Return distributions are skewed, suggesting unreliable policy quality. Heatmaps and policy plots further show scattered or indecisive strategies, especially in edge states. This highlights either poor tuning or fundamental difficulty in value estimation in this domain with the given methods.

5.2. Lunar Lander

5.2.1. PPO

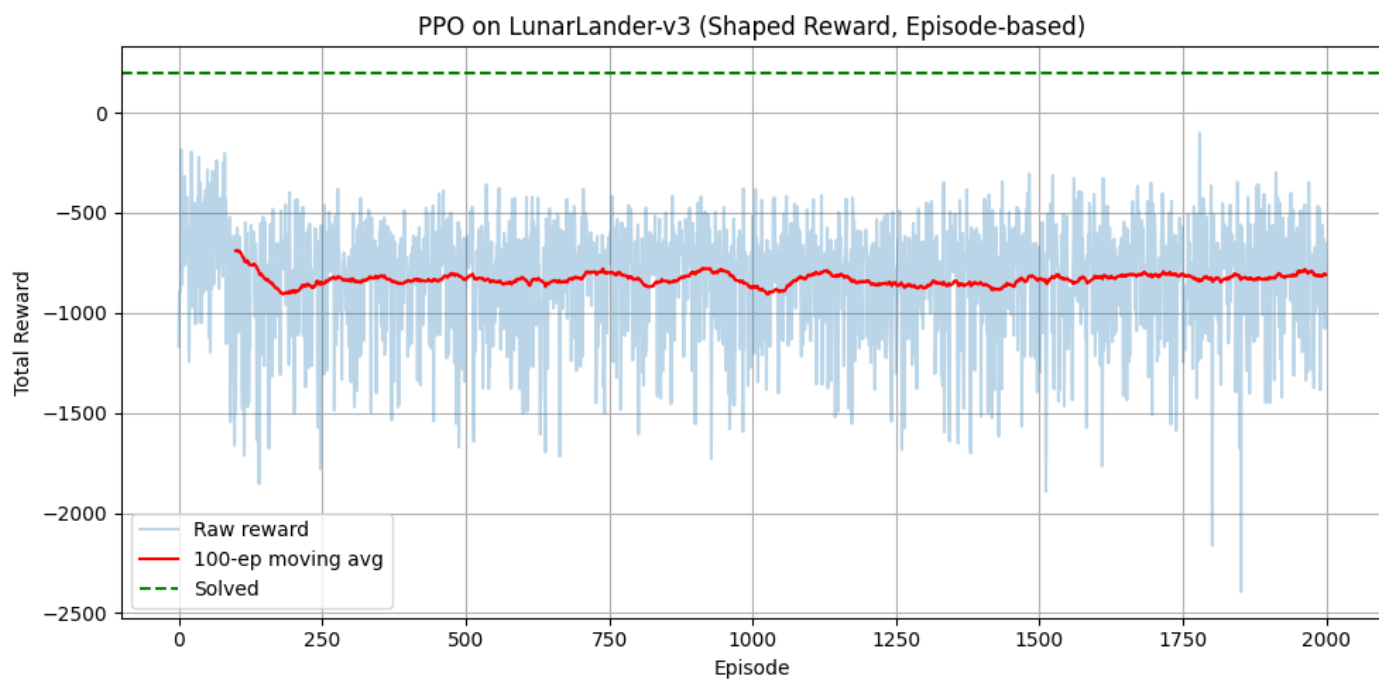


Figure 6 - PPO's Reward distribution

Rewards are broadly spread with a long tail toward negative values, suggesting PPO struggled with consistent landings and failed to converge effectively.

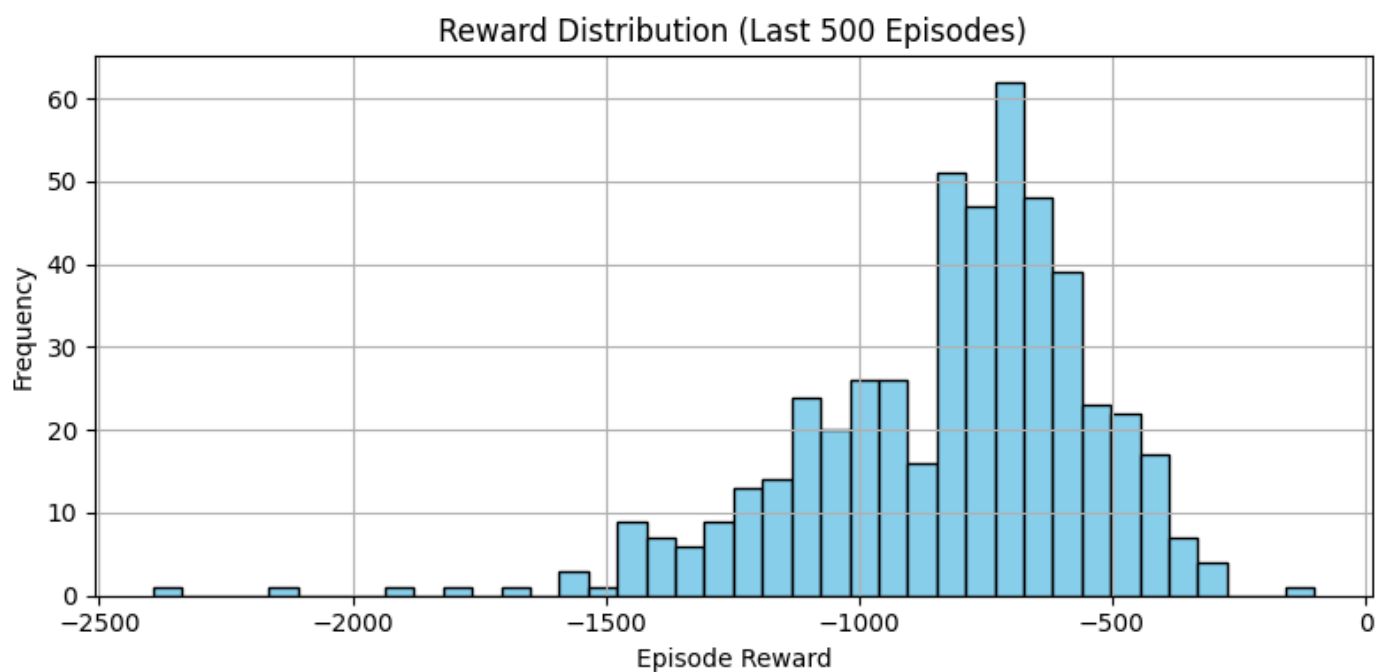


Figure 7 - PPO's Last 500 Episode's Reward Distribution

Even late in training, the distribution is still centered around low rewards, far below the 200-point threshold required to consider the task solved.

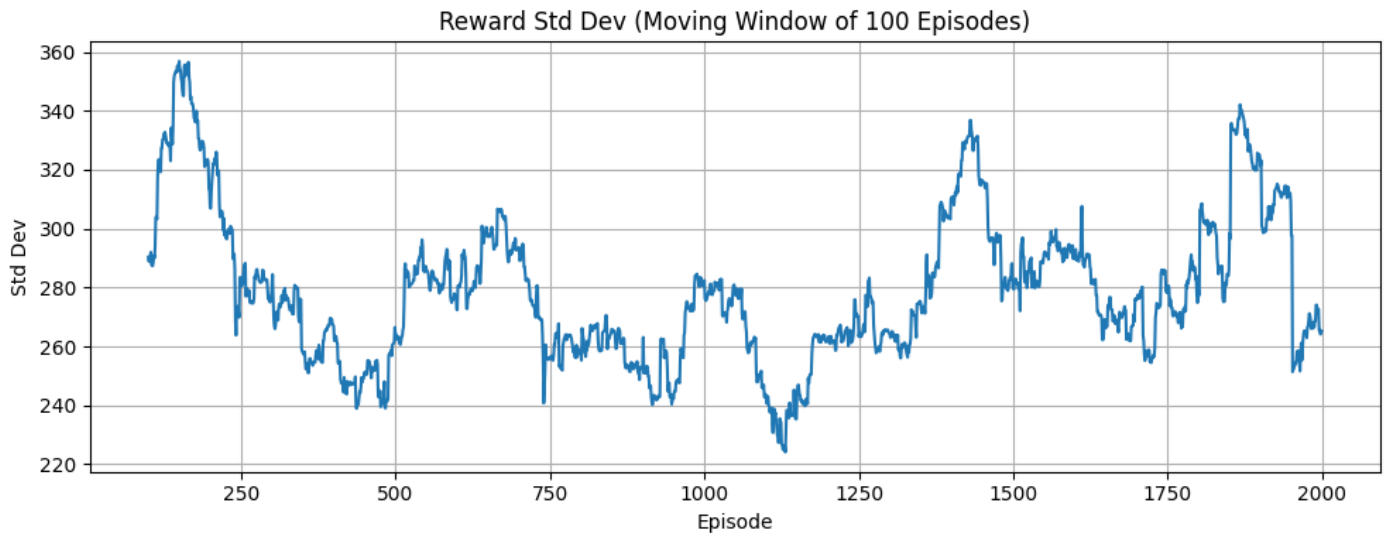


Figure 8 - PPO's Moving Average Standard Deviation

While some reduction in variability is visible (especially in the middle episodes), the persistence of high standard deviation implies PPO's policy remained unstable and inconsistent.

5.2.2. SAC

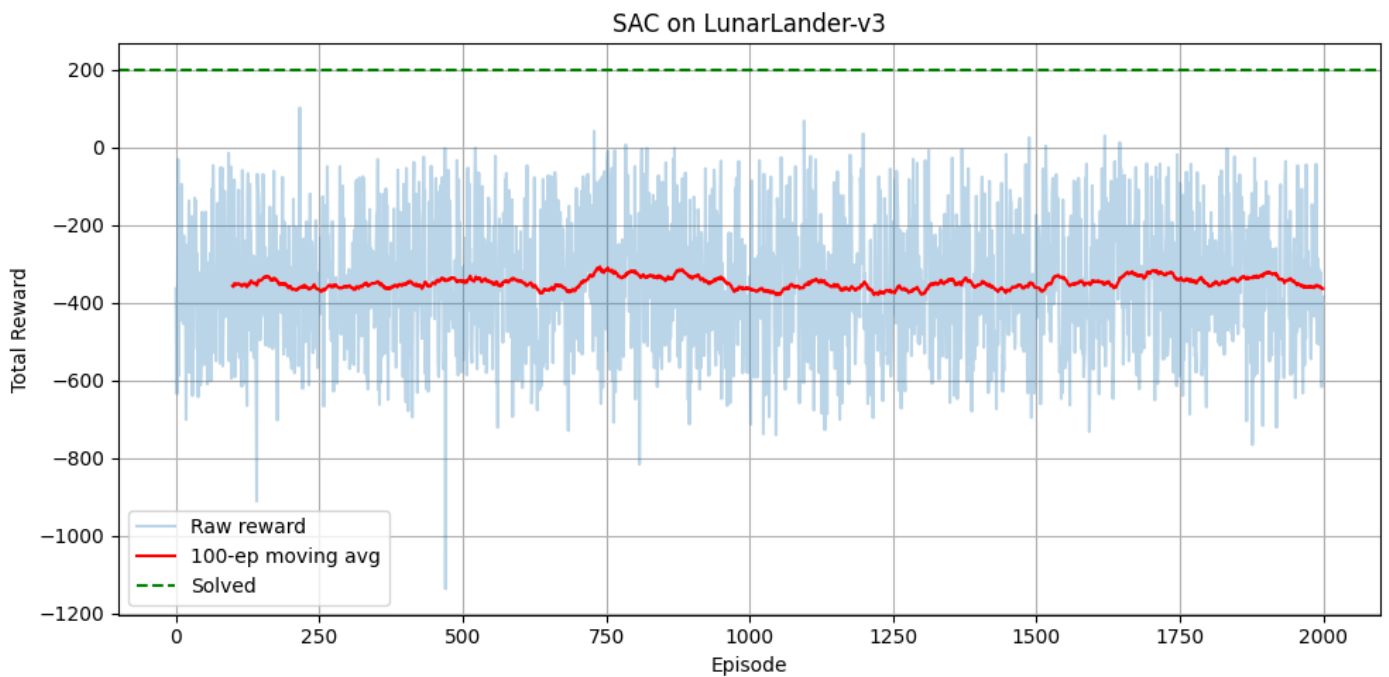


Figure 9 - SAC's Reward Distribution

SAC shows a slightly narrower spread than PPO, but the bulk of episodes still cluster around sub-optimal reward ranges, indicating poor performance overall.

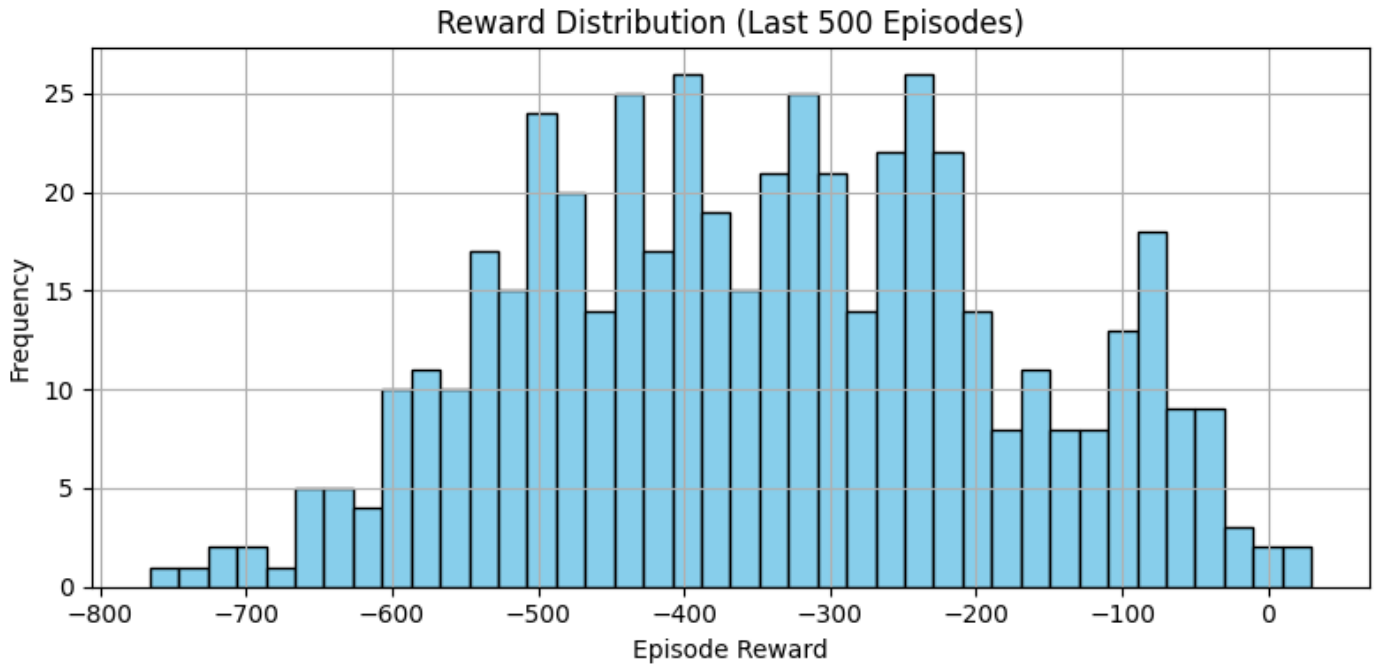


Figure 10 SAC's Last 500 Episode's Reward Distribution

No clear sign of improvement: reward values remain low, and the policy does not appear to have improved significantly over time.

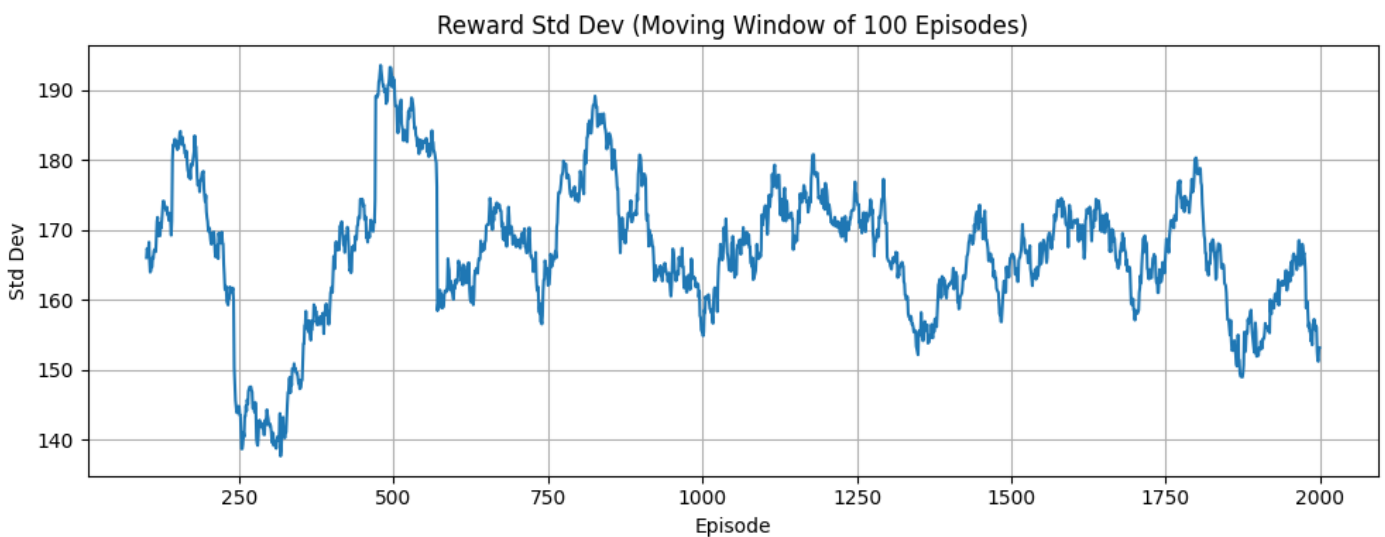


Figure 11 SAC's Moving Average Standard Deviation

Despite being slightly lower than PPO's, the variance remains high, reinforcing that SAC too failed to converge reliably.

In summary, both approaches failed with convergence never seeming close. Variables like wind power and gravity probably added too much noise and the models couldn't find solutions.

6. Conclusion

Despite algorithmic suitability, neither environment was successfully solved. High variance, environmental stochasticity, and insufficient convergence reveal challenges in applying tabular and policy gradient methods under noisy, complex dynamics.

7. References

- [1] Hado Hasselt. 2010. Double Q-learning. In *Advances in Neural Information Processing Systems*, J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta (Eds.), Vol. 23. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2010/file/091d584fced301b442654dd8c23b3fc9-Paper.pdf
- [2] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction*. MIT press.
- [3] https://gymnasium.farama.org/environments/toy_text/blackjack/
- [4] https://gymnasium.farama.org/environments/box2d/lunar_lander/
- [5] <https://xaviergeerinck.com/2020/06/07/training-the-continuous-lunar-lander-with-reinforcement-learning-rllib-and-ppo/>
- [6] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *ArXiv, abs/1707.06347*.
- [7] Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *ArXiv, abs/1801.01290*.